

DETERMINING K CLUSTERS IN K-MEANS CLUSTERING WITH THE CRAB AL-
GORITHM

A Thesis
presented to
the Faculty of California Polytechnic State University,
San Luis Obispo

In Partial Fulfillment
of the Requirements for the Degree
Master of Science in Statistics

by
Jasmine Kristine Soriano Cabrera

June 2026

© 2026

Jasmine Kristine Soriano Cabrera

ALL RIGHTS RESERVED

COMMITTEE MEMBERSHIP

TITLE: Determining k Clusters in k-Means Clustering with the
CRAB Algorithm

AUTHOR: Jasmine Kristine Soriano Cabrera

DATE SUBMITTED: June 2026

COMMITTEE CHAIR: Kelly Bodwin, Ph.D.
Professor of Statistics

COMMITTEE MEMBER: Allison Theobald, Ph.D.
Professor of Statistics

COMMITTEE MEMBER: Julia Schedler, Ph.D.
Professor of Statistics

ABSTRACT

Determining k Clusters in k-Means Clustering with the CRAB Algorithm

Jasmine Kristine Soriano Cabrera

Unsupervised clustering algorithms such as k-Means and Hierarchical Clustering are commonly used in research and academia. While many different scoring methods have been developed to determine the most optimal number of clusters (i.e. the Elbow Method, Silhouette Score, etc.), their vague results and initial assumptions about the data, present the need for a new approach in finding the optimal k number of clusters. To address this, a new cluster scoring method, the CRAB algorithm, was created. This approach works specifically with k-Means and utilizes random subsampling of the data along with the classification of its pairwise points. The constructed definition of “Buddies” and “Rivals” provides insight into which observation points tend to stick together and which tend to be further apart. For research and educational purposes, an R package is being built, implementing different functions to help understand the many capabilities of the cRab algorithm.

Keywords: Clustering, Data Science, k-Means.

ACKNOWLEDGMENTS

This paper is not only a reflection of the past four years of my education, but also all of the love and support placed in me by my family. I would first like to dedicate this to my Mom, my younger sister, Kirsten, my younger brother AJ, and most importantly, my Dad, who I miss every day.

I would also like to dedicate this to my second family found in Statistics Department, where we bonded over late nights in the Stat Lab to the Grad Lab.

TABLE OF CONTENTS

	Page
LIST OF TABLES	vii
LIST OF FIGURES	viii
CHAPTER	
1 Introduction	1
1.1 Motivation	1
1.2 Current Methods	2
1.3 Previous Research	5
1.4 cRab Score	5
2 Properties and Details	7
2.1 Overview	7
2.2 Preparing the Data	8
2.3 Buddies Matrix	9
2.4 Mean Matrix	11
2.5 Similarity Score	11
2.6 cRab Performance	12
3 The cRab Package in R	17
3.1 Package Goal	17
3.2 Design Philosophy and Ecosystem Integration	17
3.3 Functional Pipeline:	18
3.3.1 Stage I: Initialization	19
3.3.2 Stage II: Configuration	19
3.3.3 Stage III: Execution	23
3.4 Computational Strategy and Resource Management	23
3.5 Synthetic Data Generation	25

3.5.1	Point Diagnostics	28
4	Simulation Study	30
4.1	Non-Spherical Data	30
5	Conclusion	31
	REFERENCES	32
	APPENDICES	
A	simulate_mvn()	33

LIST OF TABLES

Table	Page
2.1 cRab Performance on Palmers Penguins Table Line Graph	15
3.1 Execution Stage Example	23

LIST OF FIGURES

Figure	Page
1.1 Elbow Method Graph using WCSS	2
1.2 Silhouette Score Data	4
1.3 Silhouette Score Graph	4
2.1 CRAB Algorithm	8
2.2 Buddies Matrix Data	10
2.3 Buddies Matrix Example	10
2.4 Mean Matrix Example	11
2.5 Palmers Penguins Data	13
2.6 Elbow Method on Palmers Penguins Data	14
2.7 Silhouette Score on Palmers Penguins Data	15
2.8 cRab Performance on Palmers Penguins Data	16
3.1 cRab Package Workflow Diagram	18
3.2 crab_prep() Example Output	19
3.3 crab_cols() Example Output	20
3.4 crab_params() Example Output	21
3.5 Configuration Stage Examples	22
3.6 Synthetic Data Generated from simulate_mvn()	26
3.7 Singular Synthetic Dataset Generation with CRAB Algorithm Workflow .	26
3.8 Multiple Synthetic Dataset Generation with CRAB Algorithm Workflow .	27
3.9 Point Contribution Example	29

Chapter 1

INTRODUCTION

1.1 Motivation

Unsupervised learning is a type of machine learning where data is unlabeled and the true target variable is unknown. It is commonly used to identify groups of observations, reduce dimensionality, or detect anomalies within data. While these methods can reveal meaningful structure in data, they are inherently difficult to evaluate because there is no ground truth to compare against, unlike in supervised learning.

A widely used approach within unsupervised learning is K-means, a type of clustering algorithm. This algorithm initializes k centroids in the data space and assigns each point to the nearest centroid based on Euclidean distance. The centroids are then updated as the mean of their assigned points and this process is repeated until the cluster assignments stabilize. Despite its simplicity, K-means has notable limitations. One flaw is the final clustering depends heavily on the initial placements of the centroids (Jaeger), which can lead to inconsistent or suboptimal results. Additionally, the use of Euclidean distance assumes that the clusters themselves are roughly spherical, an assumption that often fails for real-world data.

A major challenge in clustering as a whole is determining the appropriate number of clusters. Without prior knowledge, this choice can significantly affect the outcome, and different methods often suggest different answers. Two of the most commonly used techniques are the Elbow Method and the Silhouette Score.

1.2 Current Methods

The Elbow Method is a visual approach that evaluates clustering by calculating the within-cluster sum of squares (WCSS). This measures the sum of squared Euclidean distances between each point and its assigned cluster centroid for different values of k . As the number of clusters increases, the WCSS naturally decreases because smaller clusters reduce within-cluster variation. The optimal number of clusters is typically identified at the point where the decrease in WCSS slows dramatically and begins to level off, forming what is known as the “elbow”. The value of k just before this plateau is then chosen as the correct number of clusters. Although this method works well when the elbow is clearly visible, it does not always provide a clear answer (Kodinariya).

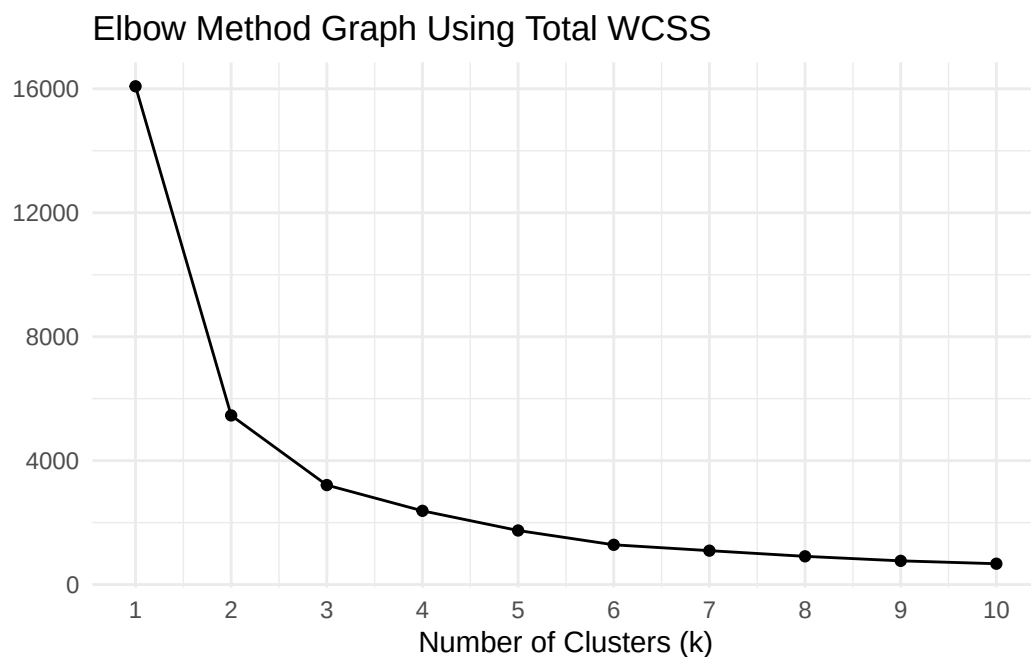


Figure 1.1: Elbow Method Graph using WCSS

Figure 1.1 illustrates a case where WCSS decreases gradually for all values of k , making it difficult to identify a distinct elbow. Without a clear plateau to guide the decision, selecting the optimal number of clusters becomes subjective and can lead to very different results depending on the value of k that is chosen.

In contrast to the Elbow Method, the Silhouette Score provides a more quantitative approach by removing the subjectivity of visual interpretation. Instead of relying on a plot, it judges clusters using a mathematical measure and selects the number of clusters based on the global maximum which corresponds to well separated and cohesive clusters. However, this method introduces its own limitation due to the assumptions it makes about cluster structure.

The Silhouette Score is defined as:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where $a(i)$ is the average distance between point i and all other points in the same cluster, and $b(i)$ is the average distance between point i and points in the nearest neighboring cluster.

Intuitively, this measure evaluates how well a point fits within its assigned cluster compared to others. Because of its formulation, the Silhouette Score favors compact and well separated clusters. As a result, it performs well when clusters are roughly uniform in shape, but struggles with oddly shaped or elongated structures that real data tends to resemble. Additionally, maximizing the global score can bias the method toward a solution with fewer clusters, prioritizing separation over accurately capturing the underlying structure. This limitation is illustrated in Figures 1.2 and 1.3.

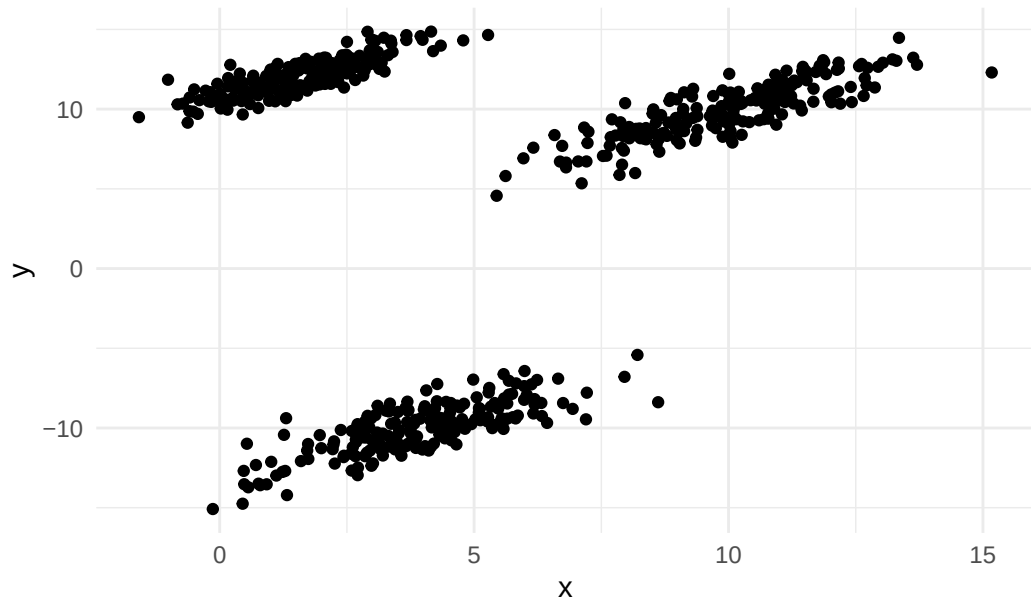


Figure 1.2: Silhouette Score Data



Figure 1.3: Silhouette Score Graph

Although the dataset visually contains three distinct clusters, the Silhouette Score identifies $k = 2$ as the most optimal. This occurs because the two upper clusters are relatively close to

each other compared to the lower cluster, leading the algorithm to favor greater separation over correctly identifying all three groups.

1.3 Previous Research

These drawbacks have motivated alternative approaches that go beyond evaluating clusters based solely on distance or compactness. One method introduced by Asa Ben-Hur and colleagues, evaluates clustering through stability. Their approach stems from the idea that a clustering into k groups captures the inherent structure if similar cluster assignments are obtained across various subsamples of the data. To assess this, the method repeatedly clusters subsets of the data using a hierarchical clustering method and compares the consistency of assignments between observations found in both subsets. Similarity between clusters is measured using cosine similarity, producing a distribution of similarity scores for each value of k . When a true underlying structure exists, these scores remain consistently high and concentrated near 1, indicating that the clustering is stable across perturbations of the data.

1.4 cRab Score

Building on this idea, the cRab algorithm evaluates clustering quality through stability rather than relying on assumptions about cluster shape or separation. By repeatedly subsampling and reclustering the data, it measures how consistent cluster assignments of pairwise observations are across different samples. This shifts the focus from how the clusters look visually to whether they are reproducible, providing stronger evidence that the identified groupings are meaningful.

Unlike the Elbow Method and Silhouette Score, the cRab score makes no assumptions about the data or group structure, allowing it to better capture more complex and irregular patterns. Additionally, the use of a global minimum provides a clear and objective selection criterion, avoiding the ambiguity of visual interpretation and the bias toward compact clusters.

Although this approach may require additional computation due to repeated sampling, it offers a more reliable way to determine the number of clusters.

The following chapters expand on this method. Chapter 2 provides a more detailed mathematical explanation, while Chapter 3 introduces an R package with functions that demonstrate the capabilities and applications of the cRab algorithm on both real and simulated data.

Chapter 2

PROPERTIES AND DETAILS

2.1 Overview

The cRab algorithm, which stands for Clustering Rivals and Buddies, is designed to evaluate clustering performance by examining how consistently points are grouped together across multiple subsamples of the data. Unlike traditional methods that rely on assumptions about cluster shape, size, or separation, cRab emphasizes reproducibility, providing a more robust measure. At its core, cRab focuses on pairwise relationships between observations. By tracking how often two points end up in the same cluster across repeated resampling, the algorithm quantifies the stability of cluster assignments. This approach allows it to handle irregular clusters that would challenge methods such as the Elbow Method and Silhouette Score. The output is a single metric, the cRab score, which provides a clear answer for selecting the most appropriate number of clusters while minimizing subjectivity.

cRab Algorithm:

Inputs:

data: dataset to be analyzed,
min_k: the starting number of clusters k for evaluation
max_k: the ending number of clusters k for evaluation
subsample_prop: proportion of the dataset to use for each resample
num_subsamples: number of resamples to generate
cluster_method: clustering algorithm

Output: A data frame where each row corresponds to a tested k cluster. The data frame includes the similarity score, runtime, and a "Best Cluster" column, which is blank for all rows except the one with the lowest cRab score, where it is marked as "BEST".

1. **For each** k value, do the following:

- Take a sample proportion of the data and perform k-Means.
- Assign Buddies and Rivals for each pair:
 - **Buddies (+1):** In the same cluster
 - **Rivals (-1):** In different clusters
 - **Ignore (0):** One or both are not in subsample
- Repeat **n** times to create n Buddies Matrices.
- Create Mean Matrix.
 - Align i, j Buddies Matrices' entries and sum to create a singular matrix.
 - Divide i, j entries by the number of comparisons between the i th and j th point.
- Get cRab score
 - Vectorize top right triangle of Mean Matrix and take the absolute value.
 - Take difference between 1 and vectorized values and square it.
 - Take the mean of the resulting values.

2. Lowest cRab Score is suggested as most optimal number of clusters.

Figure 2.1: CRAB Algorithm

2.2 Preparing the Data

At the start of the algorithm, missing observations are removed and the desired features are standardized. Standardization ensures that variables with different scales do not disproportionately influence the clustering outcome, which is especially important for distance-based

methods like K-means.

The cRab algorithm requires three main inputs:

- Subsample Proportion f : the fraction of the dataset included in each resample
- Number of resamples r : how many times the data will be subsampled and clustered
- Range of k values: the different cluster counts to evaluate

For each resample, a random subset of the data containing f proportion of the observations is drawn. K-means is then applied to assign cluster labels to each point. By repeating this process across multiple resamples, the algorithm can detect which cluster assignments are consistently reproduced, instead of placing confidence on any single clustering of the full dataset.

2.3 Buddies Matrix

After clustering each resample, a Buddies Matrix, B , is constructed to track pairwise relationships between observations. This is an $n \times n$ matrix where the entry (i, j) encodes whether points i and j were assigned to the same cluster:

$$B_{i,j} = \begin{cases} 1, & \text{if they are in the same cluster} \\ -1, & \text{if they are in different clusters} \\ 0, & \text{if one or both points are not included in the subsample} \end{cases}$$

To reduce computation and take advantage of symmetry, the diagonal and lower triangle of the matrix are set to NA. Each resample produces one Buddies Matrix, resulting in r matrices in total. Collectively, these matrices capture the variability of pairwise relationships across resamples, forming the foundation for measuring clustering stability.

For example, Figure 2.1 shows a random sample from a mock dataset clustered using K-

means. The five labeled points illustrate all possible scenarios where Observations 1 and 3 belong in different clusters, while Observations 4 and 5 are in the same cluster. Observation 2, although part of the original dataset, was not included in this particular sample.

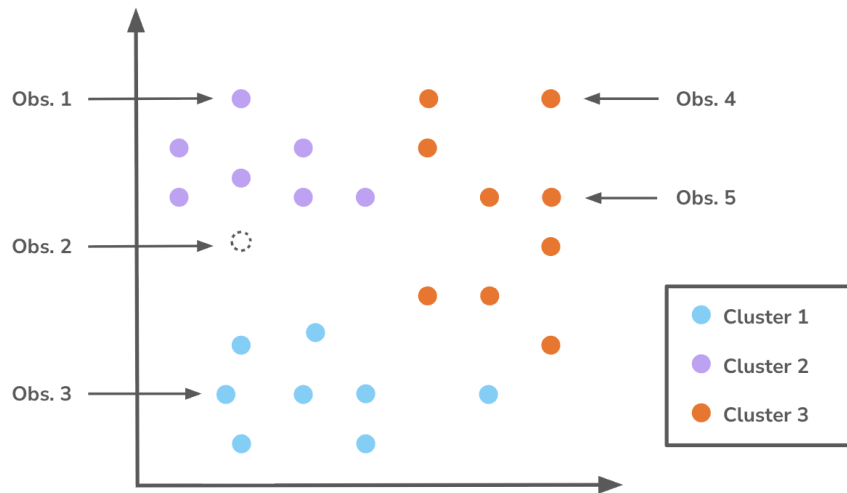


Figure 2.2: Buddies Matrix Data

Figure 2.2 shows how these pairwise relationships are represented in a Buddies Matrix.

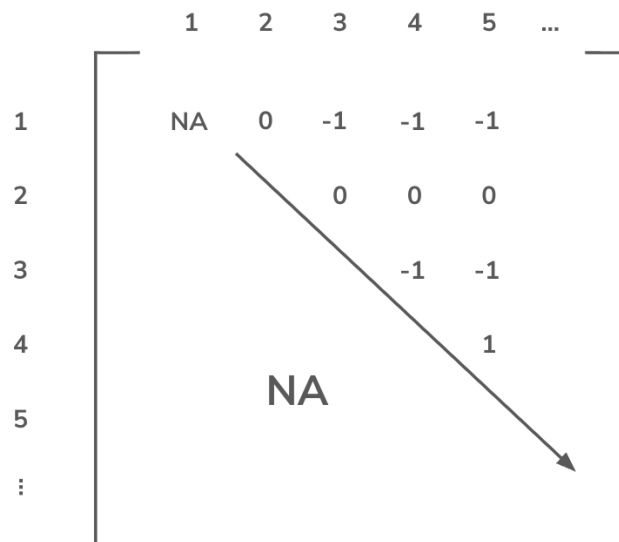


Figure 2.3: Buddies Matrix Example

2.4 Mean Matrix

The Buddies Matrices are then aggregated to form the Mean Matrix, which summarizes the overall stability of pairwise cluster assignments. For each entry (i, j) , the algorithm first counts the number of resamples where both points were included. It then sums the corresponding entries from the Buddies Matrices across those resamples, and divides by the number of comparisons to obtain the mean. The resulting entries are values between -1 and 1 that reflect how consistently the pair of points is clustered together across resamples. Values closer to -1 suggest that the points are often placed in different clusters, while values closer to 1 suggest that the points are frequently assigned to the same cluster. Entries near 0 indicate inconsistent or ambiguous pairwise assignments. The Mean Matrix acts as a stability map, revealing which relationships are robust across different subsamples of the data. Figure 2.3 is a visual diagram of how the Mean Matrix is constructed using the Buddies Matrices.

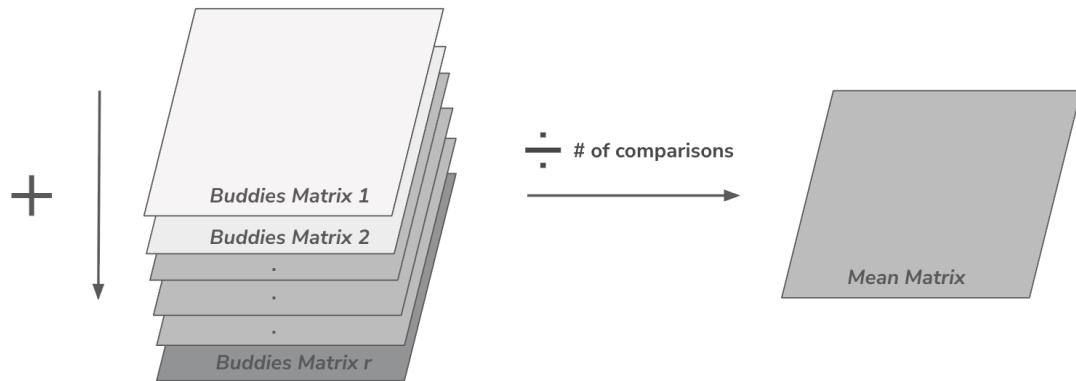


Figure 2.4: Mean Matrix Example

2.5 Similarity Score

To summarize the information in the Mean Matrix, the cRab score is calculated. All entries of the matrix are vectorized, their absolute values are taken, and the mean of the

squared distances from 1 is computed. Scores close to 0 indicate that points are consistently grouped together across all subsamples, suggesting highly reproducible clusters. In contrast, higher scores denote greater instability, signaling that cluster assignments vary significantly depending on the subsample. This score condenses these various pairwise relationships into a single metric that can be compared across different values of k .

2.6 cRab Performance

By plotting the cRab scores across different values of k , the algorithm identifies the most stable clustering as the one with the lowest score. This provides an objective criterion for selecting the number of clusters. In many cases, the optimal value appears as a “pinching point” in the plot, where the score decreases before rising again for larger k values.

To illustrate the performance of cRab, the well-known Palmer Penguins dataset serves as an effective benchmark. This dataset is particularly useful because it contains a true class label, species, allowing for direct comparison between identified clusters and the actual groupings. There are three unique species along with several physical measurements, such as bill length, bill depth, and flipper length. For this analysis, flipper length and bill length are selected, as these features provide strong separation between species with minimal overlap across observations.

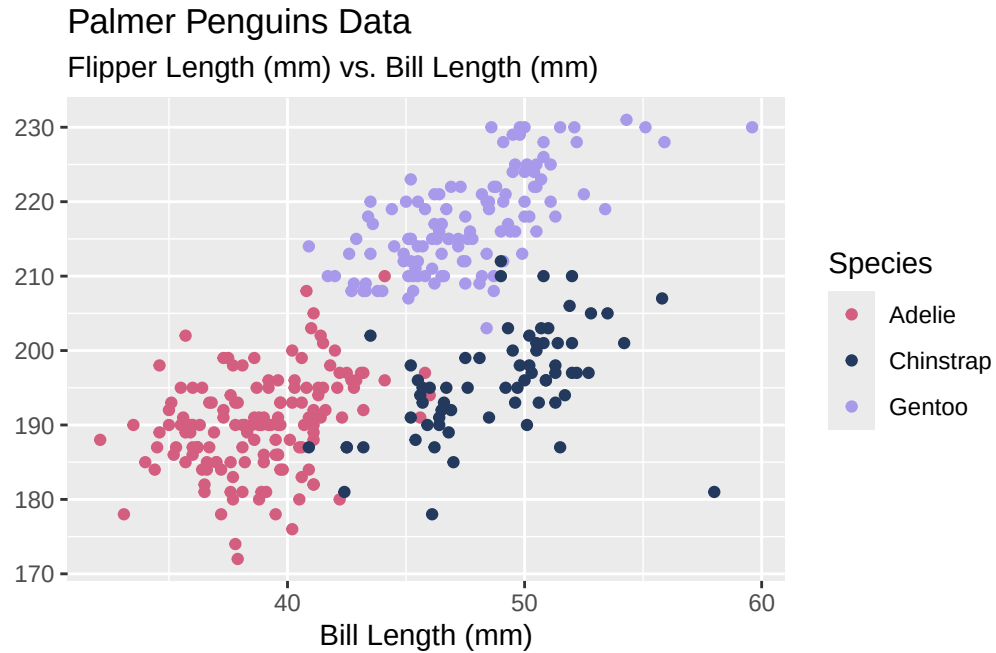


Figure 2.5: Palmers Penguins Data

When applying the Elbow Method, the resulting plot in Figure 2.5 shows a gradual decrease in WCSS rather than a distinct bend. As discussed previously, this makes it difficult to clearly identify a clear optimal number of clusters, since there is no obvious point where the reduction in error slows significantly. As a result, the method provides little guidance and requires subjective interpretation.

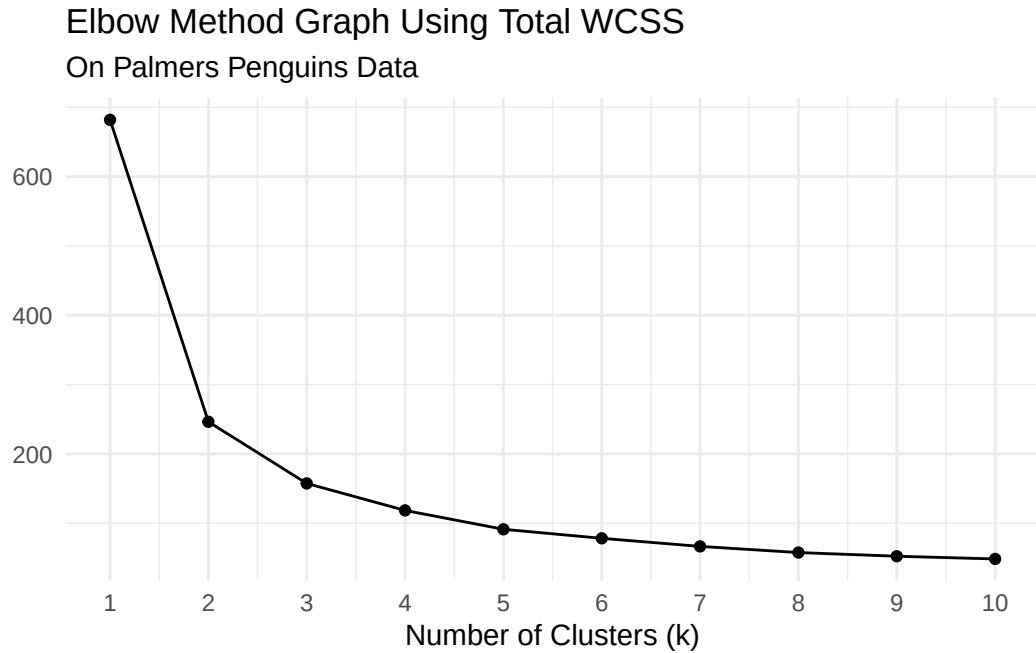


Figure 2.6: Elbow Method on Palmers Penguins Data

The Silhouette Score, shown in Figure 2.6, suggests that two clusters are optimal, as this value produces the highest average silhouette score. According to this metric, the data is best partitioned into two groups. However, this result conflicts with the known structure of the dataset, which contains three distinct species. This discrepancy highlights the method's tendency to favor solutions that maximize separation.

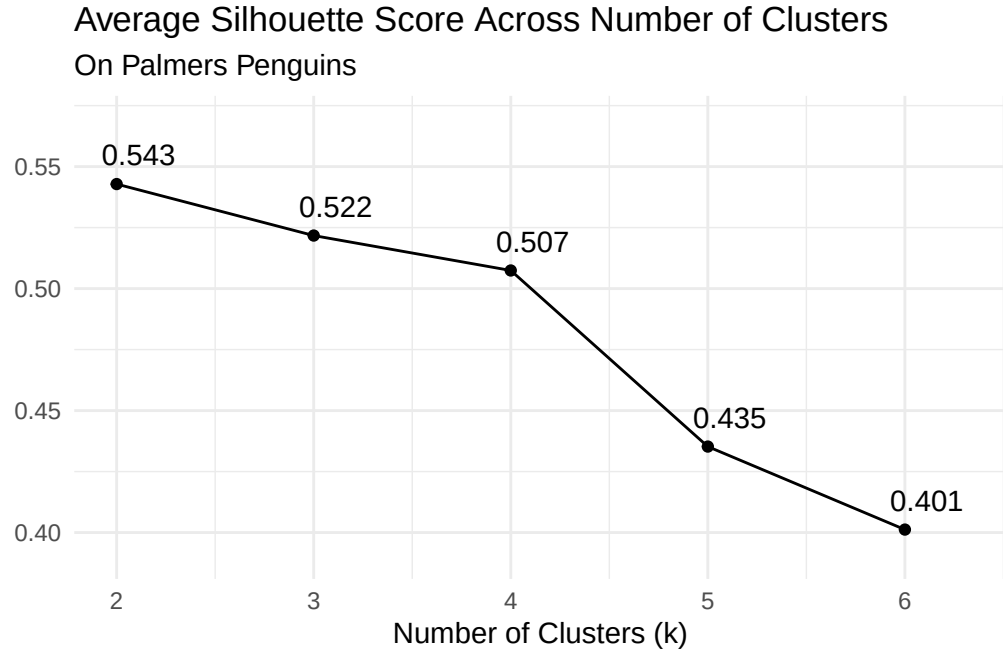


Figure 2.7: Silhouette Score on Palmers Penguins Data

In contrast, the cRab Score correctly identifies three clusters as the optimal solution with a score of 0.0048. The next lowest score occurs at $k = 2$, which is consistent with the alternative methods. Although the “pinching point” is less visually pronounced in this example, the local minimum at $k = 3$ provides a clear and objective indication of the suggested number of groupings. More importantly, this result aligns with the known structure of the dataset, as defined by the species label.

Table 2.1: cRab Performance on Palmers Penguins Table Line Graph

k	Score	Time (seconds)	Best Cluster (k)
2	0.0073200	52.255	
3	0.0048351	48.618	BEST
4	0.1260321	46.879	
5	0.0932065	42.886	
6	0.0937262	36.619	

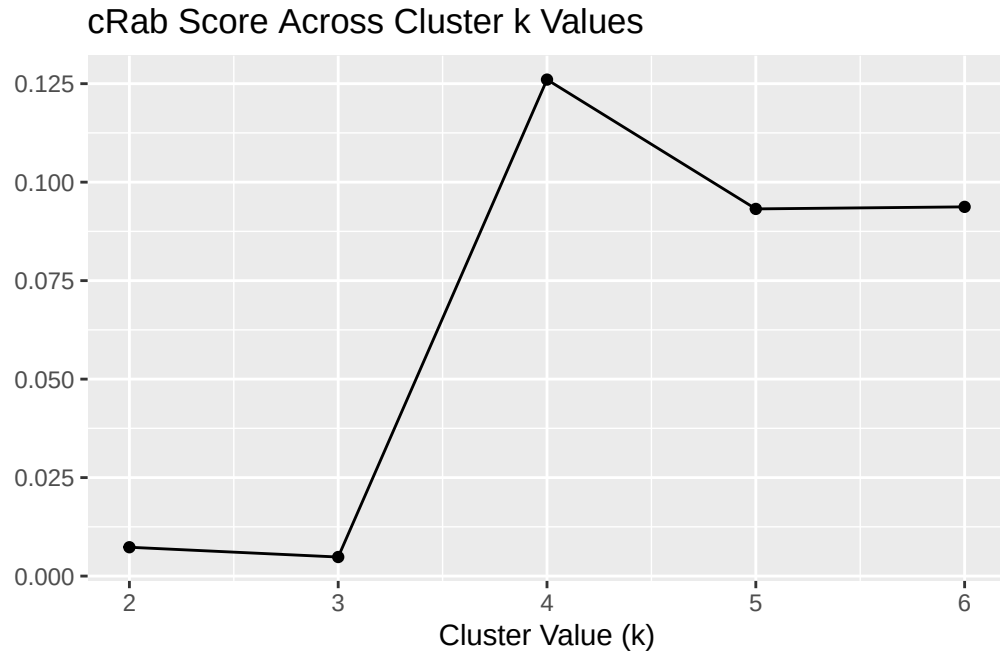


Figure 2.8: cRab Performance on Palmers Penguins Data

Chapter 4 goes more into depth through a series of simulations that demonstrate cRab's performance across various datasets, highlighting its ability to identify the true structure of the data. Boxplots and other visualizations are used to illustrate how cRab performs.

Chapter 3

THE CRAB PACKAGE IN R

3.1 Package Goal

The primary objective of the cRab package is to provide a reliable, automated framework for calculating the CRAB score within the R language. We focused on a tidy and efficient implementation that aligns seamlessly with the tools and workflows R users already employ. By formalizing this process into an accessible package, cRab offers a rigorous yet intuitive toolkit for validating cluster structures without the need for complex, custom syntax.

3.2 Design Philosophy and Ecosystem Integration

The cRab package is developed as a specialized extension of the tidyclust ecosystem (cite). By mirroring the foundational principles of the tidymodels (cite) framework, the package treats clustering as an integrated phase of a machine learning workflow rather than a standalone procedure. This architectural choice allows the package to remain familiar to modern data scientists by relying exclusively on standard R objects like tibbles and parsnip-style models. By adopting this “recipe” style of modularity, cRab fits into existing data pipelines without requiring users to learn or implement particular data formats.

The core logic of cRab is built around the native R pipe (`|>`) (cite), which establishes a baseline requirement of R Version 4.1. While this introduces a specific version constraint, it prioritizes the long-term stability and readability of the codebase. By utilizing the native pipe, the package reduces external dependencies, such as the magrittr package (cite) which houses the magrittr pipe (`%>%`), resulting in a lighter installation footprint and more transparent error logs during debugging. This modern approach ensures that the syntax remains intuitive

and sustainable as the R language continues to evolve.

The resulting user experience is both intentional and predictable. Rather than relying on a single, complex function call, cRab deconstructs the analysis into a structured progression. This naturally guides the user from raw data to a finalized similarity score while providing a structured environment that is both robust and accessible.

3.3 Functional Pipeline:

The architecture of the cRab package leverages a non-destructive, state-carrying workflow. Unlike traditional R functions that return immediate transformations of data, cRab employs a system that deconstructs the analysis into three logical stages: Initialization, Configuration, and Execution (Figure 3.1). This design allows users to make intentional decisions and review the workflow as it progresses, ensuring that metadata is preserved and can be validated throughout the entirety of the analysis.

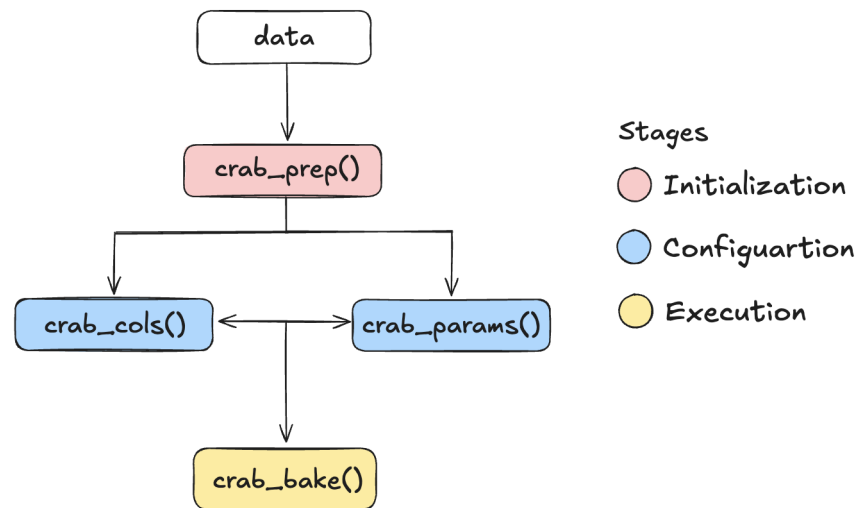


Figure 3.1: cRab Package Workflow Diagram

3.3.1 Stage I: Initialization

At the heart of the pipeline is the `crab_prep` object, which serves as the backbone and the single source of truth for the entire validation process. Upon initialization of the workflow, the package generates a metadata-rich list that captures the raw data alongside a set of empty slots reserved for subsequent variable selections and hyperparameters. By carrying the state within this object, `cRab` facilitates a precomputation phase where the data structure can be analyzed and stored. This allows for later stages of the pipeline to remain computationally efficient and internally consistent.

```
iris |>
  crab_prep()

  $data

  $original_cols
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"

  $specified_cols
NULL

  $specified_params
list()
```

Figure 3.2: `crab_prep()` Example Output

3.3.2 Stage II: Configuration

Following the initialization, the pipeline enters the Configuration Stage. To maximize flexibility and readability, `cRab` deconstructs the requirements into two modular functions: `crab_cols()` and `crab_params()`. The first function, `crab_cols()`, manages variable selection, allowing the user to define which features contribute to the distance metrics and cluster validation. This function populates the `specified_cols` element within the `crab_prep` object (Figure 3.3).

```
iris |>
  crab_prep() |>
  crab_cols(c("Petal.Length", "Petal.Width"))

$data

$original_cols
[1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

$specified_cols
[1] "Petal.Length" "Petal.Width"

$specified_params
list()
```

Figure 3.3: crab_cols() Example Output

Complementing this, crab_params() defines the hyperparameters required for the Buddies and Rivals logic, such as subsampling rates or iteration counts. This step updates the specified_params element (Figure 3.4).

```
iris |>
  crab_prep() |>
  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
             cluster_method = "kmeans")
```

```

$data

$original_cols
[1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

$specified_cols
NULL

$specified_params
$specified_params$min_k
[1] 3

$specified_params$max_k
[1] 5

$specified_params$num_subsamples
[1] 10

$specified_params$subsample_prop
[1] 0.8

$specified_params$cluster_method
[1] "kmeans"

```

Figure 3.4: `crab_params()` Example Output

These two functions are both order-independent. Consequently, a researcher can call `crab_cols()` after `crab_params()`, or vice versa, and overwrite previous specifications without compromising the object’s integrity. Each call updates a specific slot within the `crab_prep` metadata list without altering the original data. This design emulates the recipe style of tidymodels ecosystem, where instructions are layered onto an object without “baking” them into the final result.

```

# option 1
iris |>
  crab_prep() |>
  crab_cols(c("Petal.Length", "Petal.Width")) |>
  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans")

```

```
# option 2
iris |>
  crab_prep() |>
  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans") |>
  crab_cols(c("Petal.Length", "Petal.Width"))
```

```
$data

$original_cols
[1] "Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

$specified_cols
[1] "Petal.Length" "Petal.Width"

$specified_params
$specified_params$min_k
[1] 3

$specified_params$max_k
[1] 5

$specified_params$num_subsamples
[1] 10

$specified_params$subsample_prop
[1] 0.8

$specified_params$cluster_method
[1] "kmeans"
```

Figure 3.5: Configuration Stage Examples

To ensure the integrity of the `crab_prep` object before it reaches the computationally intensive Execution Stage, `cRab` implements a rigorous validation layer. Every function within the pipeline begins with an input check to verify that the object contains the necessary metadata and valid data types. Beyond simple data type checking, the package also utilizes educational warnings to guide the user. For instance, if a user specifies $k = 1$, a scenario that is mathematically trivial for clustering stability, the package does not trigger a hard stop error. Instead, it issues a guided warning that explains why $k = 1$ is statistically non-informative. This approach steers the user toward more robust and intentional practices while keeping

the code’s momentum.

3.3.3 Stage III: Execution

The transition from a specification to a result occurs exclusively during the Execution Stage via `crab_bake()`. At this point, the package reconciles the stored metadata and applies the configured logic to the raw data. This function handles the “heavy lifting” by performing the iterative subsampling and calculation of the final CRAB scores. The process outputs a tidy dataframe, providing a clear and reproducible summary of the cluster validation.

```
iris |>
  crab_prep() |>
  crab_cols(c("Petal.Length", "Petal.Width")) |>
  crab_params(min_k = 3, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans") |>
  crab_bake()
```

Table 3.1: Execution Stage Example

k	Score	Time (seconds)	Best Cluster (k)
2	0.0001581	21.969	BEST
3	0.0002594	44.644	
4	0.0941276	21.463	
5	0.1022005	20.908	

3.4 Computational Strategy and Resource Management

Due to the Buddies Matrices’ $n \times n$ structure within the cRab pipeline, as the volume of the data increases, the computational cost of cluster validation grows non-linearly. To ensure that the package is able to perform well across different local machines, it maximizes

throughput via managed parallelism and minimizes the memory footprint of its core data structures.

To accelerate the iterative Buddies and Rivals logic, cRab utilizes the doParallel and foreach libraries. While parallelization significantly reduces the total execution time by distributing iterations across multiple CPU cores, it also introduces a significant memory overhead, as each core calls for a copy of the global environment.

To mitigate the risk of system instability, the package by default, implements a conservative core-reservation policy where

$$\text{Cores}_{\text{Active}} = \max(1, \text{detectCores}() - 5)$$

By reserving a subset of cores for background system processes, the package prevents the local machine from being overwhelmed and needing to be restarted. This design choice allows the researcher to continue other tasks while the execution stage remains active in the background.

As previously mentioned, the primary computational bottleneck in cRab is the construction of the Buddies Matrix. This process operates at a complexity of $O(n^2)$ where n represents the number of observations in the subsample. As n grows, the memory required to store pairwise relationships can exceed the available Random Access Memory (RAM), leading to memory-mapping errors.

To address this, the functional pipeline utilizes a minimalist storage approach. Instead of storing redundant information, the package optimizes the memory footprint by only storing unique relationships. By utilizing the symmetry of the pairwise matrix to store only the lower triangle of indices, it effectively halves the required storage for unique relationships. Additionally, the package uses sparse indexing, ensuring that only relationships between observations currently present in a given subsample are tracked during the precomputation

phase.

By reducing the memory footprint of these structures, the cRab package allows larger datasets to be processed on standard hardware without comprising the mathematical rigor behind the CRAB score.

3.5 Synthetic Data Generation

While cRab is designed to validate clusters in real-world datasets, evaluating the package’s sensitivity can be done with a controlled dataset where the “ground truth” of the cluster structure is known. To explore this, the package includes `simulate_mvn()`, a utility that allows users to benchmark the CRAB score against datasets with specified cluster properties before applying the tool to ambiguous, and unlabeled data.

Multivariate Normal (MVN) distributions were intentionally selected for this framework because they offer precise control over cluster separation, orientation, and density. By manipulating the covariance structures of these distributions, cRab can simulate various degrees of cluster overlap, and geometric tilting.

Mechanically, `simulate_mvn()` translates these constraints into data using three primary user inputs: the total number of observations (n), the desired number of clusters (k), and a variance scalar. By adjusting these parameters, the user can purposely induce cluster overlap, or maintain clearly separated boundaries.

Synthetic Data Generated from `simulate_mvn()`

Parameters: $n = 150$, variance = 0.2, desired number of clusters = 3

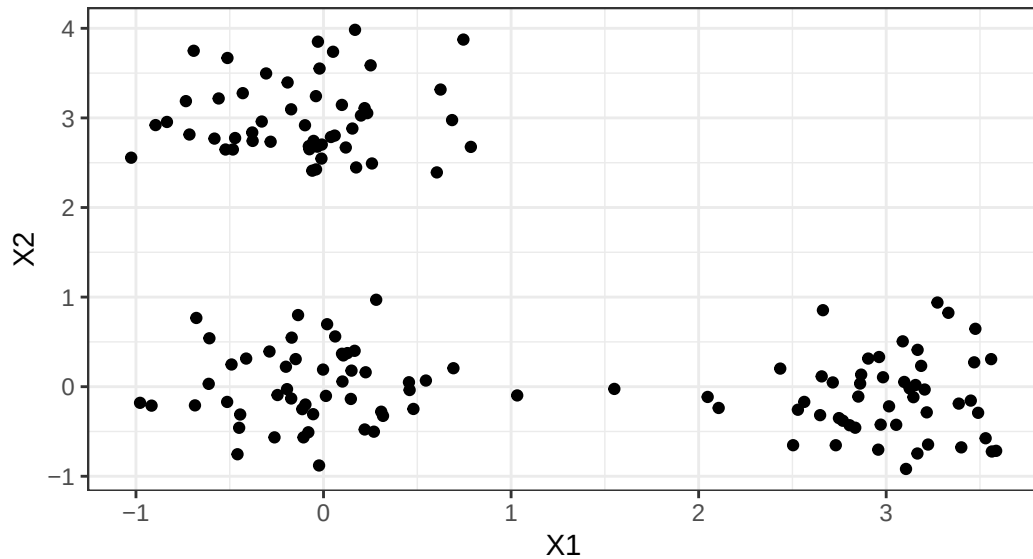


Figure 3.6: Synthetic Data Generated from `simulate_mvn()`

For a single analysis of simulated data, the workflow follows the standard pipeline (Figure 3.7). In this scenario, `simulate_mvn()` generates a static dataframe which is then passed through the standard preprocessing and parameter-specification layers.

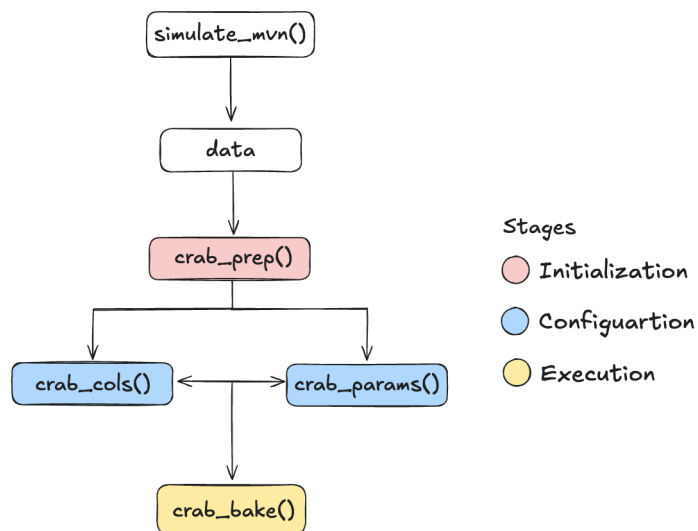


Figure 3.7: Singular Synthetic Dataset Generation with CRAB Algorithm Workflow

Below is an example code chunk:

```
# data from simulate_mvn()
data |>
  crab_prep() |>
  crab_cols(c("X1", "X2")) |>
  crab_params(min_k = 2, max_k = 5, num_subsamples = 10,
              cluster_method = "kmeans") |>
  crab_bake()
```

When evaluating the CRAB algorithm's performance across multiple iterations or varying noise levels, the pipeline shifts to a parameter focused workflow (Figure 3.8). This is outlined by two primary options. The first approach follows a similar pipeline architecture as the main pipeline. This process begins with passing `NULL` to `crab_prep()`, and calling `crab_sim_params()` to define generative parameters (i.e. `num_runs`, variance, and `n`). This is concluded with `repeat_simulations()`, which replaces `crab_bake()` and applies the CRAB algorithm to a specified number of datasets with identical parameter constraints. Alternatively, the user can avoid the pipeline and pass in all parameters into `repeat_simulations()` directly.

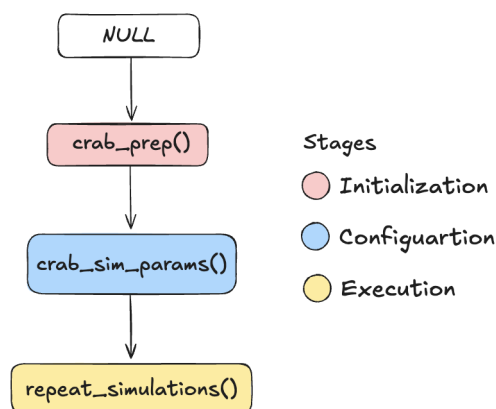


Figure 3.8: Multiple Synthetic Dataset Generation with CRAB Algorithm Workflow

To illustrate these options, consider the following implementations:

```
# option 1 -- pipeline format
crab_prep(data = NULL) |>
  crab_sim_params(min_k = 2, max_k = 5, num_subsamples = 10,
                  num_runs = 2, variance = c(0.3, 0.4),
                  cluster_method = "kmeans") |>
  repeat_simulations()

# option 2 -- directly passing in parameters
repeat_simulations(min_k = 2, max_k = 5, num_subsamples = 10,
                  num_runs = 2, variance = c(0.3, 0.4),
                  cluster_method = "kmeans")
```

Chapter 4 provides more insight into the specific performance results of these simulations.

3.5.1 Point Diagnostics

Beyond providing a global assessment of cluster stability, the cRab framework also includes specialized diagnostic tools to identify specific drivers of that stability, or rather, the sources of instability. The `point_contribution()` function allows for a granular, observation-level analysis, quantitatively and visually isolating how each individual data point impacts the final CRAB score.

In practice, raw instability values are difficult to interpret. While mathematically useful, these values can be visually overwhelming when plotted because the scale of influence varies significantly across different datasets and dimensionalities. To improve interpretability, `point_contribution()` utilizes a sigmoid transformation. This mapping projects raw contribution metrics onto a bounded 0 to 1 scale, effectively normalizing the influence of each observation.

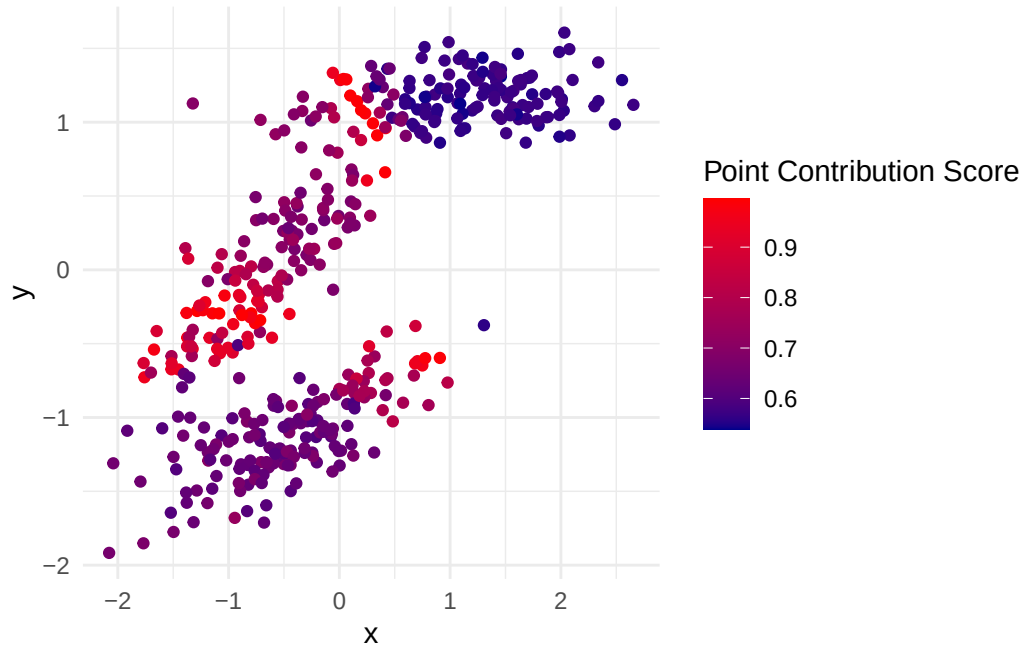


Figure 3.9: Point Contribution Example

The sigmoid transformation highlights the “swing voters” within the dataset. These observations are typically located in ambiguous regions, often between cluster boundaries, making them the most likely to switch cluster assignments across subsamples. In Figure 3.9, we see a clear example of this phenomenon. The bright red points are concentrated in the specific areas where clusters determine their boundaries. As a result, they yield a higher contribution to the final CRAB score. This function allows researchers to pinpoint the exact observations that disproportionately drive the final CRAB score.

Chapter 4

SIMULATION STUDY

test w non spherical data too

4.1 Non-Spherical Data

Chapter 5

CONCLUSION

- restate and brag
- #better than everybody else
- future work
- note that this was only used for k-means ## Computational Details

The results in this paper were obtained using R 4.4.1. R itself and all packages used are available from the Comprehensive R Archive Network (CRAN) at <https://CRAN.R-project.org/>.

REFERENCES

APPENDICES

Appendix A

SIMULATE_MVN()

```
# enter functions here
```